JavaScript

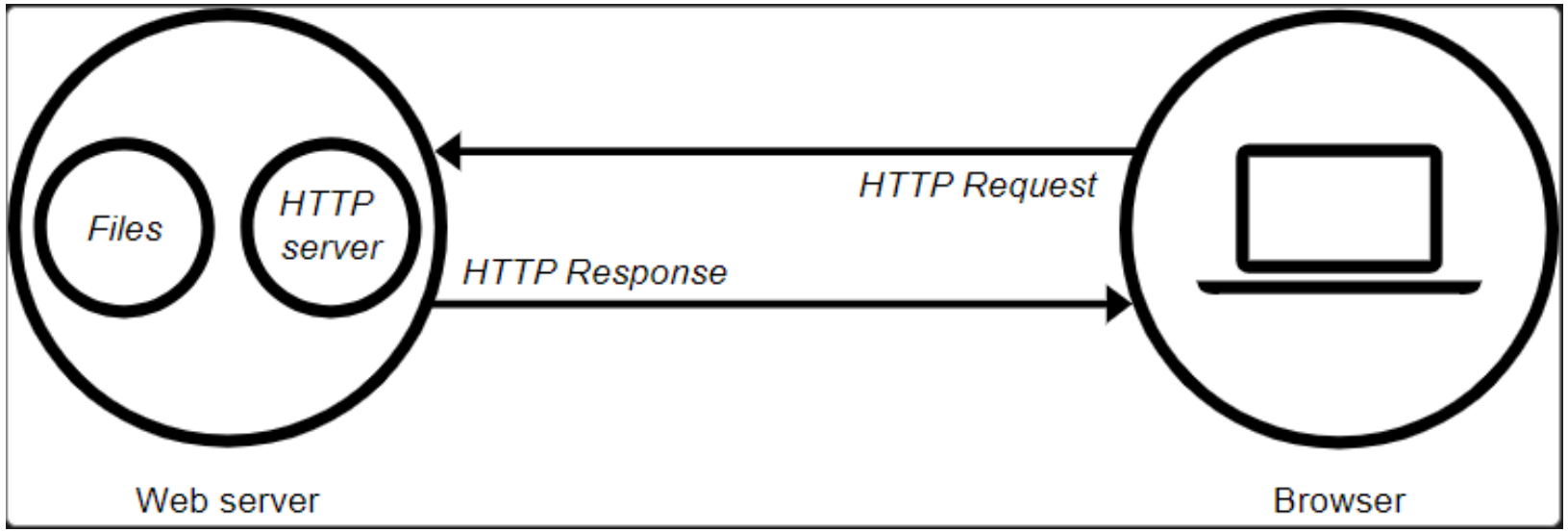
- JavaScript is the programming language of the Web.

- Client side interpreted scripting language

  -To program the behavior of web pages

- It is now possible to create entire web applications in which both client-side and server-side logic is written in JavaScript.

Web server - Client



JavaScript

- On a web server, the HTTP server is responsible for p answering incoming requests.
- Upon receiving a request, an HTTP server first checks URL matches an existing file.
- If so, the web server sends the file content back to the browser. If not, an application server builds the necessary file.
- If neither process is possible, the web server returns an error message to the browser, most commonly 404 Not Found. The 404 error is so common that some web designers devote considerable time and effort to designing 404 error pages.

What can JavaScript do?

- JavaScript gives HTML designers a programming tool
- JavaScript can be used to validate data
- JavaScript can read and write HTML elements
- Create pop-up windows
- Perform mathematical calculations on data
- React to events such as a user rolling over an image or clicking a button
- This allows you to write cross-platform apps in a really easy way.
- Retrieve the current date and time from a users computer or the last time a document was modified
- JavaScript can put dynamic text into an HTML page
- JavaScript can be used to create cookies

JavaScript Advantages?

- Less Server Interaction
- Immediate feedback to the visitors
- Increased interactivity
- Richer interfaces

JavaScript is case-sensitive

Each line of code is terminated by a semicolon

Embedding JavaScript in HTML file

```
<html>
        <head>
                <script language="javascript" type="text/javascript">

                        document.write("Hello World!")

                </script>
        </head>
        <body>

        </body>
</html>
```

Comments in JavaScript

- Any text between a **//** and the end of a line is treated as a comment and is ignored by JavaScript.
- Any text between the characters **/* and */** is treated as a comment. This may span multiple lines.

Javascript Alerts

```
<script language = "javascript" type = "text/javascript">
        alert(“Hello World!”);
</script>
```

Javascript Display Posibilities

- Writing into an HTML element, using innerHTML.
  - **document.getElementById("demo").innerHTML = “HELLO”;**
- Writing into the HTML output using document.write().
  - **document.write(“WORLD”);**
- Writing into an alert box, using window.alert().
  - **window.alert(“Window alert”);**
- Writing into the browser console [For debugging purposes], using console.log()
  - **console.log(“console log”);**

Popup boxes in Javascript

- alert
- confirm
- prompt

Programming elements of JavaScript

- Variables, Datatypes, operations
- Statements
- Arrays
- Functions
- Objects in JavaScript
- Exception Handling
- Event
- Dynamic HTML with javaScript

Variables

There are 4 ways to create JavaScript variable

- Using var
- Using let
- Using const
- Using nothing

Variables

- The var keyword is used in all JavaScript code from 1995
- Variable creates with **var** statement

  var <variable name> = <some value>
- Variable name must start with a letter (a to z or A to Z), underscore(_), or dollar($) sign.
- After first letter we can use digits (0 to 9), for example value1.
- We can not use spaces in names.
- Variable names are case sensitive

var

```
var num = 0;
var str;

str = “hello”;
num = 3;
```

let

- Added to JavaScript in 2015.
- If you think the value of the variable can change, use let
- Can not be re-declared

const

- Added to JavaScript in 2015.
- If you want a general rule: always declare variables with const.
- Must be assigned
- Can not be re-declared
- Can not be reassigned

```
const price1 = 5;
const price2 = 6;
let total = price1 + price2;
```

Javascript Data Types

- Primitive data type: String, Number, Boolean, Undefined, Null
- Non-Primitive data type: Object, Array, RegExp

JavaScript Operators

- Arithmetic Operators
- Comparison (Relational) Operators
- Bitwise Operators
- Logical Operators
- Assignment Operators
- Special Operators

Arithmetic Operators

Operator	Description	Example
+	Addition	10+20 = 30
-	Subtraction	20-10 = 10
*	Multiplication	10*20 = 200
/	Division	20/10 = 2
%	Modulus (Remainder)	20%10 = 0
++	Increment	var a=10; a++; Now a = 11
--	Decrement	var a=10; a--; Now a = 9

Comparison Operators

Operator	Description	Example
==	Is equal to	10==20 = false
===	Identical (equal and of same type)	10==20 = false
!=	Not equal to	10!=20 = true
!==	Not Identical	20!==20 = false
>	Greater than	20>10 = true
>=	Greater than or equal to	20>=10 = true
<	Less than	20<10 = false
<=	Less than or equal to	20<=10 = false

Bitwise Operators

Operator	Description	Example
&	Bitwise AND	(10==20 & 20==33) = false
|	Bitwise OR	(10==20 | 20==33) = false
^	Bitwise XOR	(10==20 ^ 20==33) = false
~	Bitwise NOT	(~10) = -10
<<	Bitwise Left Shift	(10<<2) = 40
>>	Bitwise Right Shift	(10>>2) = 2
>>>	Bitwise Right Shift with Zero	(10>>>2) = 2

Logical Operators

Operator	Description	Example
&&	Logical AND	(10==20 && 20==33) = false
||	Logical OR	(10==20 || 20==33) = false
!	Logical Not	!(10==20) = true

Assignment Operators

Operator	Description	Example
=	Assign	10+10 = 20
+=	Add and assign	var a=10; a+=20; Now a = 30
-=	Subtract and assign	var a=20; a-=10; Now a = 10
*=	Multiply and assign	var a=10; a*=20; Now a = 200
/=	Divide and assign	var a=10; a/=2; Now a = 5
%=	Modulus and assign	var a=10; a%=2; Now a = 0

Special Operators

Operator	Description
(?:)	Conditional Operator returns value based on the condition. It is like if-else.
,	Comma Operator allows multiple expressions to be evaluated as single statement.
delete	Delete Operator deletes a property from the object.
in	In Operator checks if object has the given property
instanceof	checks if the object is an instance of given type
new	creates an instance (object)
typeof	checks the type of object.
void	it discards the expression's return value.
yield	checks what is returned in a generator by the generator's iterator.

if - else

```
if(expression1){
        //content to be evaluated if expression1 is true
}
else if(expression2){
        //content to be evaluated if expression2 is true
}
else if(expression3){
        //content to be evaluated if expression3 is true
}
else{
        //content to be evaluated if no expression is true
}
```

switch

```
switch(expression){
        case value1:
                //code to be executed;
                break;
        case value2:
                 //code to be executed;
                break;
        ......

        default:
                //code to be executed if above values are not matched;
}
```

function

A JavaScript function is a block of code designed to perform a particular task.

```
function name(parameter1, parameter2, parameter3) {
        // code to be executed
}
```

JavaScript loops

- for loop
- while loop
- do-while loop
- for-in loop

for loop

```
for (initialization; condition; increment)
{
    //code to be executed
}
```

while loop

```
while (condition)
{
   //code to be executed
}
```

Do while loop

```
do{
   //code to be executed
}while (condition);
```

Exercise JavaScript

- Write a JavaScript program to find the area of a rectangle where lengths and with of its sides are 7, 3.

- Write a JavaScript program to calculate multiplication and division of two numbers (input from user)

The Document Object Model (DOM)

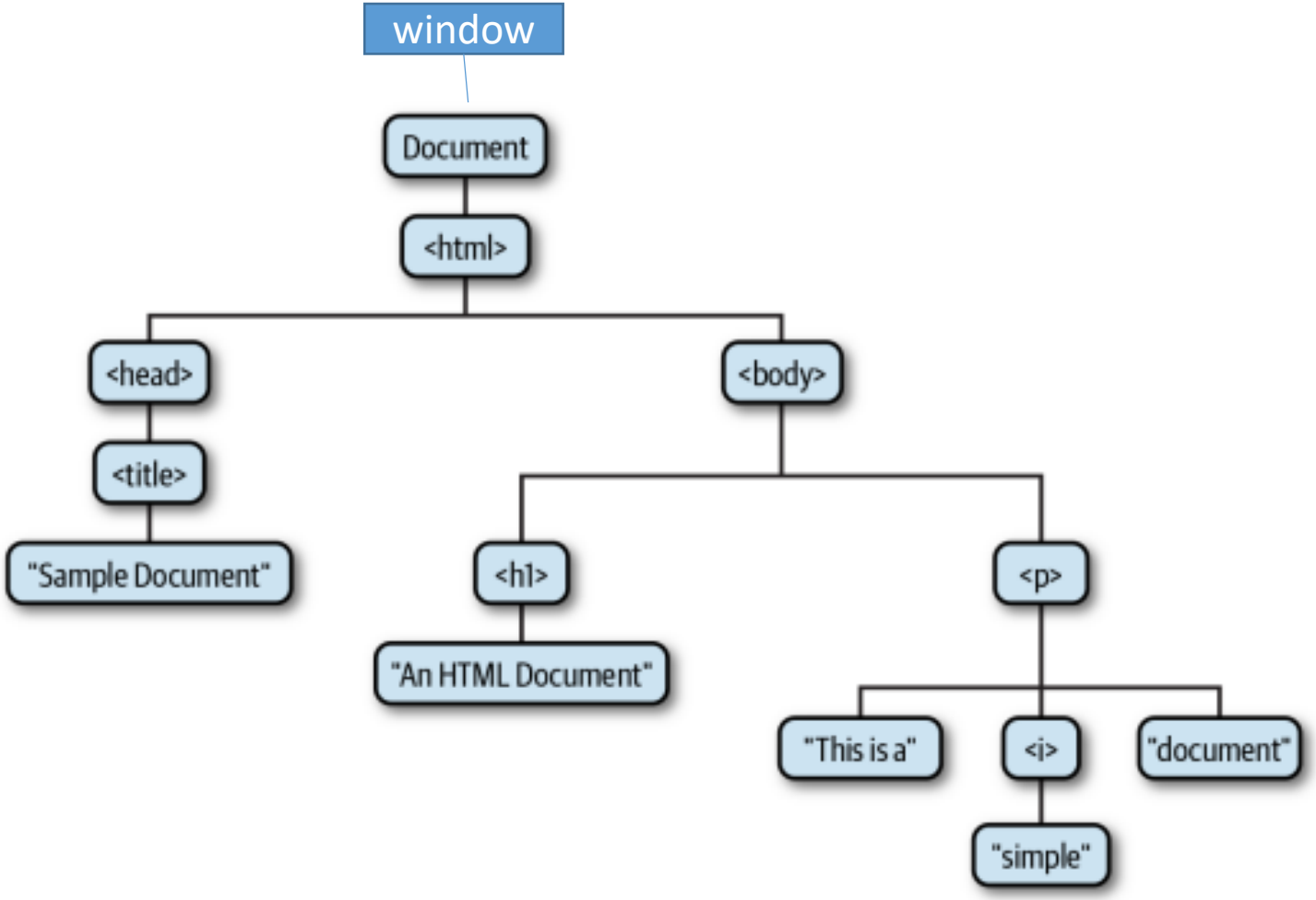
- No browser sees content or source code on a page as people do.

- One of the most important objects in client-side JavaScript programming is the Document object which represents the HTML document that is displayed in a browser window or tab.

- Reference: https://developer.mozilla.org/en-US/docs/Web/API/Document_Object_Model/Introduction

Tree representation of an HTML Document

```
<html>
  <head>
    <title>Sample Document</title>
  </head>
  <body>
    <h1>An HTML Document</h1>
    <p>This is a <i>simple</i> document.
  </body>
</html>
```



DOM

- There is a JavaScript class corresponding to each HTML tag type, and each occurrence of the tag in a document is represented by an instance of the class.
- The tag, for example, is represented by an instance of HTMLBodyElement, and a <table> tag is represented by an instance of HTMLTableElement.

- https://www.w3.org/2003/01/dom2-javadoc/org/w3c/dom/html2/HTMLBodyElement.html

- https://developer.mozilla.org/en-US/docs/Web/API/HTMLBodyElement
- https://developer.mozilla.org/en-US/docs/Web/API/HTMLTableElement

Why DOM

- Can change all the HTML elements in a web page
- Can change HTML attributes in a page
- Can change CSS styles in a page
- Can add or remove HTML elements or attributes and etc.

Methods of Document Object

Method	Description
write("string")	writes the given string on the doucment.
writeln("string")	writes the given string on the doucment with newline character at the end.
getElementById()	returns the element having the given id value.
getElementsByName()	returns all the elements having the given name value.
getElementsByTagName()	returns all the elements having the given tag name.
getElementsByClassName()	returns all the elements having the given class name.

Creating table using JavaScript DOM

- Step1:

  HTML:

  <button onclick="generateTable()"> Generate Table </button>

- Step2:
  - Javascript
  - Function -> generateTable()

```
// get the reference for the body
  var body = document.getElementsByTagName("body")[0];

// creates a <table> element and a <tbody> element
  var tbl = document.createElement("table");
  var tblBody = document.createElement("tbody");
```

Creating table using JavaScript DOM

```
// creating all cells
for (var i = 0; i < 2; i++) {
        // creates a table row
        var row = document.createElement("tr");
        for (var j = 0; j < 2; j++) {
                // Create a <td> element
                var cell = document.createElement("td");
                //Create a text node, make the text
                var cellText = document.createTextNode("cell in row "+i+", column "+j);
                // node the contents of the <td>, and put the <td> at the end of the table row
                cell.appendChild(cellText);
                row.appendChild(cell);
   }
 // add the row to the end of the table body
   tblBody.appendChild(row);
 }
```

Creating table using JavaScript DOM

```
// put the <tbody> in the <table>
tbl.appendChild(tblBody);
// appends <table> into <body>
body.appendChild(tbl);
// sets the border attribute of tbl to 2;
tbl.setAttribute("border", "2");
```

JavaScript Events

- Client-side JavaScript programs are almost universally event driven: rather than running some kind of predetermined computation.

- They typically wait for the user to do something and then respond to the user's actions.

- The web browser generates an event when the user presses a key on the keyboard, moves the mouse, clicks a mouse button, or touches a touchscreen device.

JavaScript Events

Event	Description
onchange	An HTML element has been changed
onclick	The user clicks an HTML element
onmouseover	The user moves the mouse over an HTML element
onmouseout	The user moves the mouse away from an HTML element
onkeydown	The user pushes a keyboard key
onload	The browser has finished loading the page

onclick()

```
<button onclick="console.write('Thank you');"> Please Click </button>
<button onclick="this.innerHTML = Date()">The time is?</button>
<button onclick="displayDate()">The time is?</button>


<script>
function displayDate() {
        document.getElementById("demo").innerHTML = Date();
}
</script>
<p id="demo"></p>
```

onload()

```
Hello <img src="white_rose.png" onload="alert(‘Rose flower loading')"/>
```